1 – Introduction to Deep Learning

Building, training, and judging *a Feed-Forward Neural Network*

1 Objectives

This lab has a two-fold objective: (i) introduce you to `PyTorch` and to how neural networks behave on a real classification task; and (ii) give you a first, non-graded experience of an AI & Cybersecurity lab and its report, as practice for the graded ones.

Unlike a tutorial, this lab is about **judgement, not recipes**. You will rarely be given a filled-in configuration. Instead you will be given a *goal*, some *constraints*, and a *simple recommended starting point*, and you will have to **decide, justify, and defend** your choices against what you actually observe in your own experiments. Concretely, you will:

- Use `PyTorch` to build and train feed-forward neural networks for **intrusion detection**.
- See how training breaks when a single fundamental (feature scaling, learning rate) is wrong, and learn to read the symptoms.
- Handle class imbalance, and reason about *which* metric matters for an intrusion detector.
- See how a single feature can secretly drive a model, and how a data-cleaning choice can silently reshape your classes.
- Build **a simple baseline**, then test whether extra complexity (depth, optimizers, etc.) is actually worth it.

A note on AI assistants. You may use AI coding assistants. They are good at writing a training loop; they are *not* a substitute for the thinking this lab grades. Every hypothesis, every justification tied to *your* curves and reports, your reading of the failure modes, and your logbook are yours to produce and to defend. A result you cannot explain – or a run you cannot justify having spent – is **worth nothing** here, whether a human or a tool produced it.

2 Lab Rules

Each student must join a self-organized group of 4 students. Each group submits **four zip files**, one per lab, named `groupID_labX.zip`. Each zip contains:

- **A report** (max 10 pages) describing your approach, decisions, experiments, and results, with tables and plots. Name it `groupID_labX.pdf`. The report is judged mainly on *reasoning* (see the rubric), not on raw performance numbers.
- **Your experiment logbook** (a section/appendix of the report or a separate sheet – see "How to work"). This is required, and does not count in the number of pages.
- **The notebook(s) and code** you wrote.
  - *Best practice:* use Markdown headers and comments to (i) understand tomorrow what you did today and (ii) make us follow your solution.
  - **The notebook must be executed**: code *and* outputs must be visible.
  - **It must run** if we re-execute it. Set seeds where you rely on reproducibility.
  - **Format:** include the `.ipynb` and an HTML export.

Notice:

- The **Lab 1 report does not contribute** to the final grade, but you **must submit it** for your other labs to be corrected.
- Submitting Lab 1 before **01/12/2026** can yield an **extra point** for the exam.
- Other reports contribute as described on the teaching portal. You may submit the four reports up to two weeks before any exam session.

3 Dataset: CICIDS2017 (subset)

The CICIDS2017 dataset (Canadian Institute for Cybersecurity, University of New Brunswick) is a benchmark for intrusion detection, combining realistic benign traffic with common attacks. More info: Kaggle and UNB.

You are given a **subset**: one CSV of labelled network flows. Each row is a flow described by a handful of flow-level features, with a `Label` column taking

one of four values: **Benign, PortScan, DoS Hulk, Brute Force**. The feature set has been reduced for you (flow durations and inter-arrival times, packet-length statistics, byte/packet rates, a few TCP-flag counts, `Total Fwd Packets`, `Destination Port`, etc.). We do not list the class balance or the issues and inconsistencies in the data here on purpose: inspecting them yourself is Task 1.

4 How to work: logbook & budget

1. The experiment logbook (required deliverable)

Keep **one logbook for the whole lab** – a single table, with one row for **every run that trains a model**. Tag each row with the task it belongs to:

Run	Task	What I changed (one thing)	Hypothesis *before* running	Key result (val)	What I concluded
1	2A	baseline: 1 hidden layer, 64 units, ReLU, lr=5e-4	PortScan easy, Brute Force hard (rare)	macro-F1 0.xx; BF recall low	imbalance hurts BF as predicted
2	2B	lr 5e-4 → 0.5	too high, loss will oscillate	val loss climbed, never settled	confirmed; keep lr moderate

Only runs that train a model go in the logbook. Preprocessing and data-analysis decisions (Task 1) belong in your report text, not here.

- **Write the hypothesis *before* you run.** A wrong hypothesis you then explain is worth *more* than a vague right one. Do not rewrite history.
- **Change one thing per run.** If you change two things at once, you can't tell which one caused the effect.

2. The run budget

You have a budget of **∼25 logged training runs for the entire lab**. Spend the budget deliberately: before each run, your logbook must say *why this run is worth one of your remaining slots.*

The runs here are cheap to compute – the budget is not about saving your laptop. It is the skill we're building: in real settings, each experiment costs money, time, and attention, and disciplined experimentation is what separates an engineer from someone trying things at random.

3. Predict-then-check

For each experiment, **predict the outcome first** (i.e., your hypothesis), then reconcile it with what happened.

5 Task 1 – Preprocessing

Before any modelling, **look at the data and decide how to prepare it.** Report what you saw and *why* you did what you did.

1. **Inspect.** Report the number of samples, the classes and their balance, and any data quality issues you find (missing values, duplicate rows, implausible/degenerate values). A couple of plots or tables are enough.
2. **Missing values & duplicates.** You will find a few missing values and a number of duplicate rows. Decide what to do with each, and justify.
   - **Q:** How many samples do you have before and after your cleaning?
3. **Split** into train / validation / test. You decide the ratios and whether to stratify.
   - **Q:** Given what you found about class balance, should the split be stratified? Why? Where did you fit any scaler/normalizer – on which partition – and what goes wrong if you fit it on all the data?
4. **Scaling.** Look at the ranges and tails of the features. Choose a scaling strategy and justify it **from the distributions you observed** [1].
   - **Q:** Which scaling did you pick and why? How did the outliers inform the choice?

No training runs are spent in this task. No need to track in the logbook.

[1] Threat `Destination Port` as a numerical feature.

6 Task 2 – Your first network

6.1 Task 2A – Build a minimal working classifier

Design a **single-hidden-layer** feed-forward network for the 4-class problem and write a clean training loop including loss, optimizer, and early stopping on the validation set.

- You choose: hidden width, activation, batch size, learning rate, stopping criterion.
- **Recommended simple starting point**: one hidden layer of 64 units, ReLU, cross-entropy loss, AdamW, learning rate `5e-4`, batch size 64, up to ∼100 epochs with early stopping. State every choice.

Then:

- **Predict first:** before training, which classes do you expect to be hardest, and why? (Tie this to what you measured in Task 1.)
- **Report the training/validation loss** curves and the validation classification report.
  - **Q:** How do you pick the "best" model across epochs? Do the curves converge?
  - **Q:** Do your hardest-class predictions hold up? Explain the per-class results.

6.2 Task 2B – Why the fundamentals matter

Your Task 2A model trained because you made sensible choices about feature scaling and learning rate. Here you'll see what those choices were actually buying you, by getting **one fundamental wrong at a time** and watching how training breaks.

Take your Task 2A setup and run these three variants. For each, **predict the loss curve first**, then run it, plot training/validation loss, and **name the failure mode and explain the mechanism** (why that choice produces that curve):

- **S1 – no feature scaling:** train on the *raw, unscaled* features (everything else as in 2A).
- **S2 – far too high learning rate:** increase it until the training breaks down. How high are you expecting it the too-high LR to be?
- **S3 – learning rate far too low**: e.g. ∼100× smaller.

Then:

- **Q:** Rank S1–S3 by how badly they hurt, and say which fundamental – scaling or the learning rate – you would get right *first* on a new problem.
- **Q (reading symptoms):** Suppose you were handed just *one* of these loss curves, with no note of what had been changed. What in the shape of the curve would point you to the cause?

Takeaway: your 2A baseline gets all three fundamentals right, which is exactly why it works. Most training problems are fundamentals like these; fancy optimizations are secondary – see Task 8.

6.3 Task 2C – Lock in your baseline

Settle on a **simple, well-tuned single-hidden-layer model you fully understand.** This is your **"baseline to beat"** for the rest of the lab.

- Report its validation *and* test classification reports.
- **Q:** Compare validation vs test. Does it generalize (is the gap small)?
- As one controlled experiment, switch the activation (e.g. Linear $\leftrightarrow$ ReLU) on this baseline. **Q:** Does it change per-class behaviour? Would it be fair to compare these two on the *test* set, or is the test set off-limits for choosing between them? Why?

7 Task 3 – When a feature biases the model

In your data, **every Brute Force flow uses destination port 80.**

- **Q:** Is "Brute Force always on port 80" a property of brute-force attacks in general, or an artefact of *this* dataset? If your model sees this, what is it likely to learn?

Counterfactual (predict-then-check). In the **test set only**, change the destination port of the **Brute Force** flows from `80` to `8080`. *Predict first:* what will happen to Brute Force recall? Then run inference with your Task 2C baseline (no retraining) and show the classification report.

- **Q:** Did your prediction hold? Explain what this reveals about what the model was really using to detect Brute Force.

Now remove the `Destination Port` feature entirely from the original data and redo your preprocessing (clean, split, scale). **Use this port-free dataset for the rest of the lab.**

- **Q:** Count your classes after cleaning now, and compare to before. One class shrinks *dramatically* when you remove the port and de-duplicate. Which one, and by how much?
- **Q:** *Why* does that class collapse while the others barely move? What does that tell you about what the model "knew" about that class before you removed the port?
- **Q:** After dropping the port, which class is now the **rarest**? How balanced is the data now?
- **Q:** Knowing that de-duplication decimates one class once the port is gone, do you still de-duplicate? Argue your choice.

8 Task 4 – The rare class: loss and imbalance

Retrain your baseline on the port-free data.

- **Predict first:** which class will suffer most now, and why? Train, show the per-class report, and confirm or revise.
  - **Q:** Can you still detect the rarest class at all?

Now use a **class-weighted** cross-entropy loss: (`CrossEntropyLoss(weight=...)`; you can get weights from `sklearn.utils.class_weight.compute_class_weight(..., "balanced")`).

- **Q:** From which partition do you compute the class weights, and why would using any other partition be wrong?
- **Predict first:** what will happen to overall *accuracy* vs *macro-F1* when you add the weights? Retrain and measure.
  - **Q:** Did accuracy and macro-F1 move in the same direction? For an intrusion detector that must catch rare attacks, **which metric should you optimize, and why?** Defend it.

Note how much a simple change (reweighting the loss) buys you here, before any architectural complexity.

9 Task 5 – Varying the hyper-parameters

Now the recurring question: **can a more complex model beat your simple port-free baseline from Task 4 on validation?** This whole

task shares a budget of about **10 runs** – spend them deliberately, with a hypothesis before each.

9.1 5A – Depth

Design **2–3 deeper candidates** (e.g. 2–4 hidden layers; widths you choose). Do **not** run a grid – *choose* configurations and justify *why*.

- **Q:** Does depth help on this problem? If a single-hidden-layer baseline stays competitive, what does that suggest about the complexity of this (low-dimensional, tabular) task?

9.2 5B – Batch size

Pick **three** batch sizes that bracket a hypothesis (e.g. 1, 64, 1024). *Predict first* the effect on (a) generalization and (b) wall-clock time.

- **Q:** Report validation results *and* training time, and explain both.

9.3 5C – Optimizer vs. learning rate

We told you that **a good learning rate often matters more than the choice of optimizer.** Here you *test that claim yourself.*

- Within your remaining budget, design a small experiment comparing SGD, SGD+momentum, and AdamW – vary the learning rate!
- **Q:** What did you find – does the optimizer or the learning rate dominate on this data?

9.4 5D – Verdict

Take the best model you found and report it on the **test** set, **side by side with your simple port-free baseline from Task 4.**

- **Q:** Did the added complexity beat the simple baseline? Was it worth the runs it cost? Answer honestly from your logbook.

10 Task 6 – Optimizations: last bit to tackle

Train this network on the port-free data: 5 hidden layers of {256, 128, 64, 32, 16} units, ReLU, cross-entropy, AdamW, lr `5e-4`, batch size 128, ∼150–200 epochs, no regularization and no early stopping. *Predict first:* what will the loss curves look like?

- **Q:** Is it overfitting? Show exactly where you see it in the curves.

Now optimize it. Try the simplest thing first, and change only one thing per run (predict-then-check; ∼6 runs for this task):

1. **First, make the model smaller or stop earlier.** Try a smaller net, or early stopping.
2. **Add one regularizer at a time:** weight decay (AdamW `weight_decay`), then dropout, then batch normalization. Check each in the validation set before adding the next.

- **Q:** Which change helped the most for the least effort?
- **Q:** In the end, did the big network — even with regularization — do better on the **test** set than your simple port-free baseline from Task 4?

Final reflection

Looking across your whole logbook: **which decisions gave you the most performance per unit of effort?** Argue it from your own runs.

11 Self-assessment Evaluation Process

Lab 1 is not graded, but this rubric reflects what we expect from your lab reports. Use these points to evaluate your submission and check whether it covers all the required aspects before submitting your report and code.

The key point is: ***working code without reasoning does not demonstrate the competence you have acquired; a sound explanation and well-structured reasoning, even with non-optimal results, is more valuable.***

The labs are not about finding the best-performing solution, and we do not compare performance across students. We want to assess ***YOUR understanding of the subject – not that of an LLM.***

Criterion	What you should provide
Logbook discipline	State your hypotheses *before* observing the results; change one element at a time; keep and discuss wrong predictions; document the experimental budget and justify how you used it.
Justified decisions	Explain and justify your choices regarding preprocessing (NaN/duplicates, split, stratification, scaling), architecture, class-weight partitioning, and the de-duplication dilemma using evidence from your experiments.
Reading failure modes (Task 2B)	Identify the three failure modes, explain the mechanism behind each of them, provide a sensible ranking, and relate the shape of the observed curves to their likely cause.
Interpretation tied to own results	Support your claims using your *own* curves, reports, and numerical results (e.g., the port-80 collapse, the PortScan collapse, or differences between accuracy and F1). Avoid relying only on generic textbook explanations.
Judgement & simplicity	Avoid unnecessary brute-force experimentation; compare added complexity against the baseline and provide a reasoned judgement on whether the additional complexity was worthwhile, particularly in Tasks 5D and 6.
Report & reproducibility	Provide a clear report within the page limit; ensure that the notebook executes correctly and that random seeds are set whenever reproducibility is claimed.